

Project Report

ShipmentSure Predicting On-Time Delivery Using Supplier Data

1. Introduction:

Dataset Name: On-Time Delivery Dataset

Source: Kaggle

Dataset Link: [On-Time Delivery Dataset](#)

Target Column: Reached.on.Time_Y.N (Binary → 0 = Late, 1 = On-Time)

2. Data Loading & Cleaning:

Steps Taken:

- Imported libraries: pandas, numpy, matplotlib, seaborn.
- Loaded dataset using `pd.read_csv()`.
- Checked **column names, data types, non-null counts** with `.info()`.
- Checked dataset `.shape()` and first few rows.
- Summary statistics with `.describe()`.
- Checked for missing values; none were found.
- Checked duplicates; removed where applicable.

Notes/Observations:

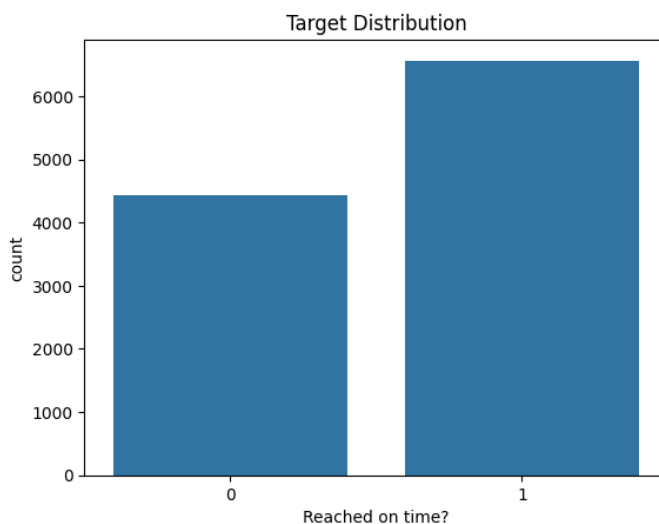
- Dataset is clean, no missing values or duplicate rows were detected.
- Numerical columns have reasonable ranges (e.g., Customer_rating between 1–5).
- Ready for analysis and preprocessing.

3. Univariate Analysis

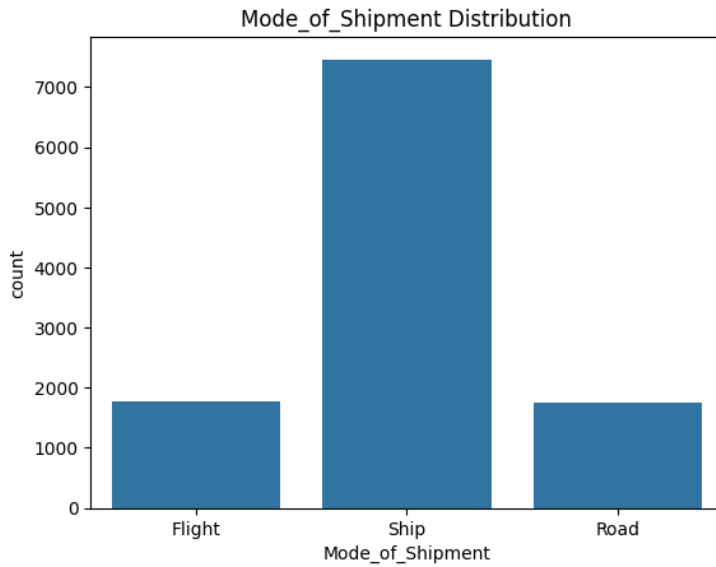
3.1 Categorical Features

- Countplots created for: Reached.on.Time_Y.N, Warehouse_block, Mode_of_Shipment, Product_importance, Gender.

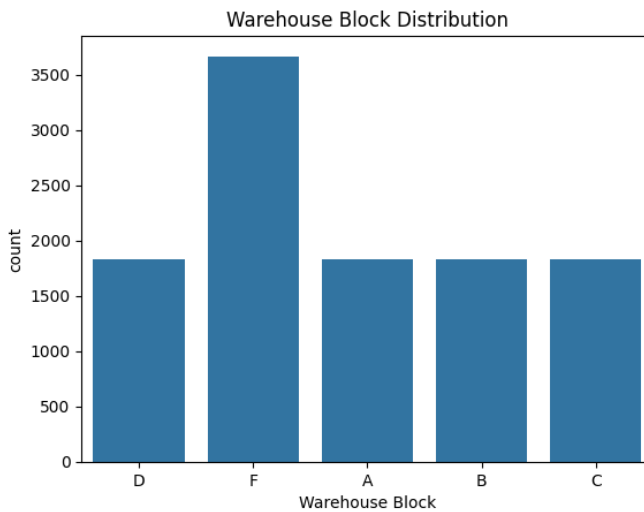
Notes/Observations:



- Majority of deliveries are **on-time** (slightly imbalanced target: ~65% on-time, ~35% late).



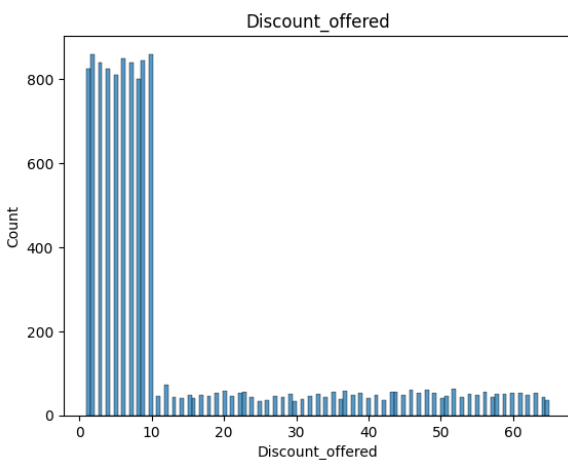
- Most shipments use **Flight** or **Ship**, while fewer shipments are sent via road .
- **Product importance** is mostly Medium.



- Warehouse blocks **D and B** handle more shipments.
- Gender distribution is nearly balanced.

3.2 Numerical Features

- Histograms and boxplots created for: Customer_care_calls, Customer_rating, Cost_of_the_Product, Prior_purchases, Discount_offered, Weight_in_gms.



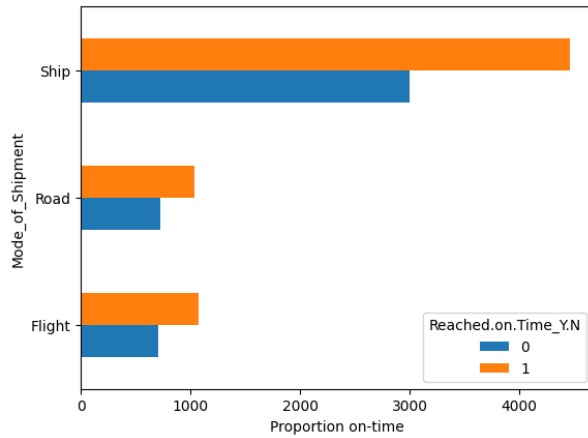
Notes/Observations:

- Customer_care_calls mostly range 1–4; few customers called more.
- Outliers observed in Cost_of_the_Product and Discount_offered.
- Weight_in_gms mostly below 2000g; some very heavy products exist.

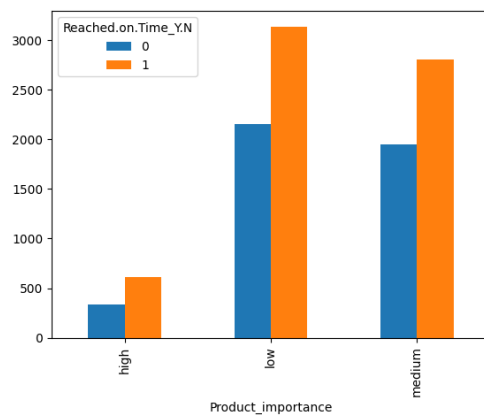
- Customer_rating mostly 3–5.

4. Bivariate Analysis:

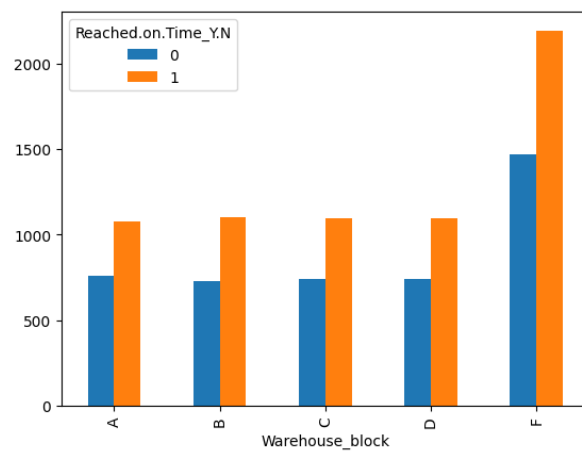
- Crosstabs and bar plots for:



- Mode_of_Shipment vs Reached.on.Time_Y.N

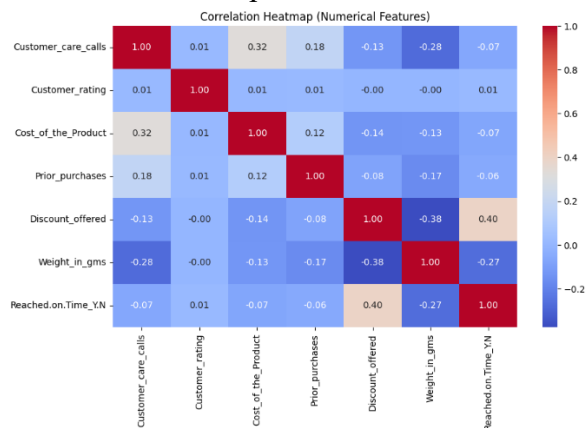


- Product_importance vs Reached.on.Time_Y.N

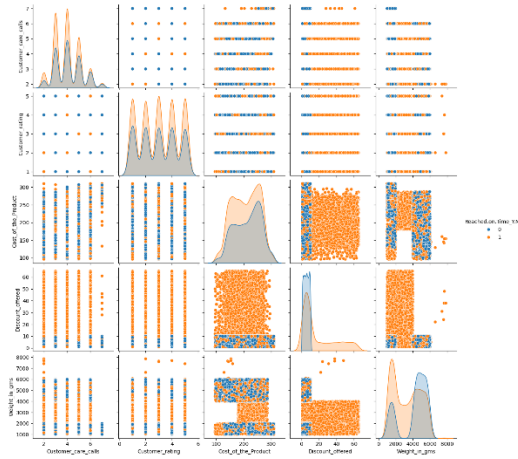


- Warehouse_block vs Reached.on.Time_Y.N

- Correlation heatmap for numerical features.



- Pairplot for numerical features colored by target.

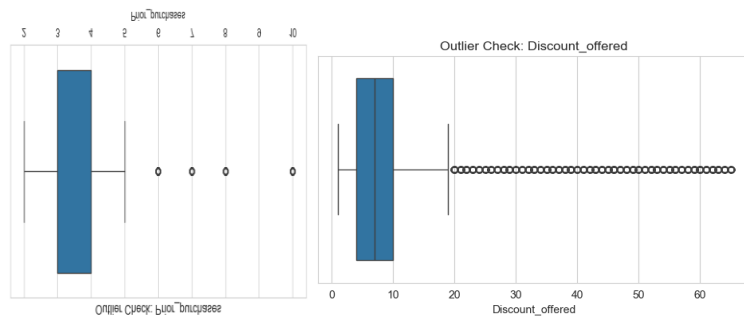


Notes/Observations:

- **Mode of shipment affects delivery:** Shipments via Ship are more delayed than those sent by Flight.
- High importance products are mostly delivered on time.
- Weak correlation observed between numerical features and target.
- No strong multicollinearity detected between numerical features.

5. Outlier Detection:

- Boxplots plotted for all numerical features.



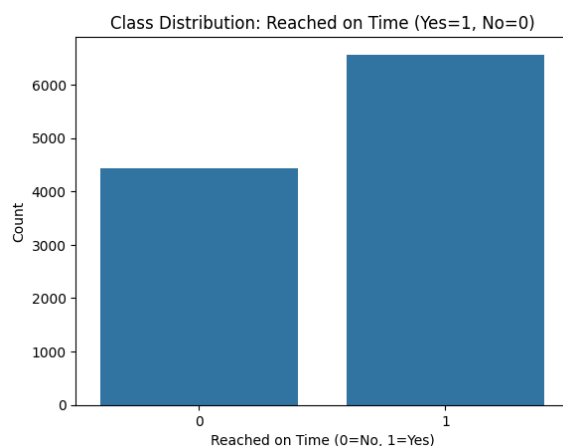
Notes/Observations:

- Outliers found in Discount_offered, and Weight_in_gms.
- Extreme values may affect some models. They can be handled through transformations or left unchanged when using tree-based models.

6. Class Imbalance:

- Count and percentage of target variable (Reached.on.Time_Y.N) calculated.
- Countplot for visualizing class distribution.

Notes/Observations:



- The target variable is slightly imbalanced: approximately 65% on-time deliveries vs. 35% late deliveries

7. Handling Missing and Infinite Values

Steps Taken:

7.1 Checked for Missing Values:

- Used `data1.isnull().sum()` to identify any missing (NaN) values across all columns.
- The dataset showed no missing values in any column initially.

7.2 Checked for Duplicates:

- Used `data1.duplicated().sum()` to identify duplicate rows.
- Found and removed duplicates using `data1.drop_duplicates()` to ensure data quality.

7.3 Handling Infinite Values:

- During the creation of the `Cost_to_Weight_ratio` feature, some infinite values appeared due to division by zero (when `Weight_in_gms` = 0).
- These were handled by replacing infinite values (`np.inf` and `-np.inf`) with NaN using:

```
encode_data.replace([np.inf, -np.inf], np.nan, inplace=True)
```

Notes/Observations:

- The dataset had no initial missing values.
- Infinite values generated during feature engineering were successfully handled.
- All NaN values were replaced, ensuring a clean dataset before encoding and scaling.

8. Encoding:

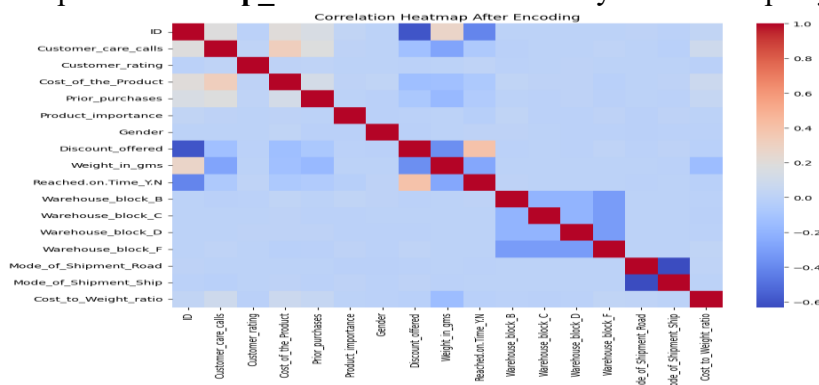
Steps Taken:

8.1 Label Encoding:

- The categorical columns '**Product_importance**' and '**Gender**' were encoded using `LabelEncoder` from `sklearn.preprocessing`.
- Label encoding converts string labels into numerical format (e.g., 'Low' → 0, 'Medium' → 1, 'High' → 2).
- This method is suitable when the categorical variable has ordinal meaning (like Product Importance).

8.2 One-Hot Encoding:

- The nominal categorical columns '**Warehouse_block**' and '**Mode_of_Shipment**' were encoded using `pd.get_dummies()`.
- Since these features are non-ordinal, one-hot encoding creates binary columns (0 or 1) for each unique category.
- The parameter `drop_first=True` avoids dummy variable traps by dropping one category.



8.3 Data Type Conversion:

After encoding, all boolean columns (True/False) were converted into numeric format (0/1) using `.astype(int)`.

9. Normalization:

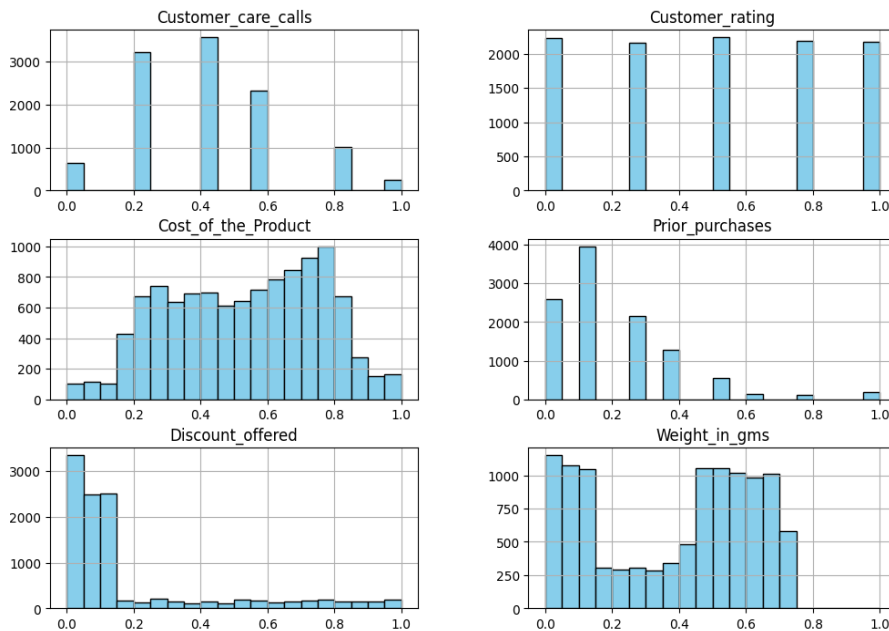
Steps Taken

- The numerical features **Customer_care_calls**, **Customer_rating**, **Cost_of_the_Product**, **Prior_purchases**, **Discount_offered**, and **Weight_in_gms** were normalized using `MinMaxScaler`.
- `MinMaxScaler` scales values to a range between **0** and **1** using the formula:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- This helps models that are sensitive to feature magnitude (like Logistic Regression, KNN, and Neural Networks) perform better.

Distribution of Scaled Numerical Features (After Normalization)



Notes/Observations:

- All categorical columns were successfully converted into numeric format.
- No categorical (object-type) columns remained after encoding.
- All numerical columns were scaled between 0 and 1 for consistency.

10. Feature Engineering:

Features Created:

1. $\text{Cost_to_Weight_ratio} = \text{Cost_of_the_Product} / \text{Weight_in_gms}$
2. $\text{Cost*Weight} = \text{Cost_of_the_Product} \times \text{Weight_in_gms}$
3. $\text{Discount_Ratio} = \text{Discount_offered} / \text{Cost_of_the_Product}$

Steps Taken

Purpose:

- These new features help capture relationships between product cost, weight, and discounts affecting delivery time.
- **Cost_to_Weight_ratio** shows cost efficiency per gram of shipment.
- **Cost*Weight** reflects combined cost–weight influence on delivery delay.
- **Discount_Ratio** indicates how the level of discount impacts the likelihood of on-time delivery.

Handling Missing and Infinite Values:

- Checked for infinity values (np.inf or $-\text{np.inf}$) that arise when $\text{Weight_in_gms} = 0$.
- Replaced infinity values with NaN using:

```
encode_data.replace([np.inf, -np.inf], np.nan, inplace=True)
```

- Filled all NaN values with the **median** of the valid ratios to maintain central tendency.

Normalization:

- Applied **MinMaxScaler** again on the new column **Cost_to_Weight_ratio** to scale it between 0 and 1.
- Ensures that the engineered feature is on the same scale as other numeric features.

Visualization:

- Plotted a histogram for **Cost_to_Weight_ratio** using **matplotlib** to view its distribution.

- Most values were concentrated near lower ranges, indicating that many products have low cost per gram.

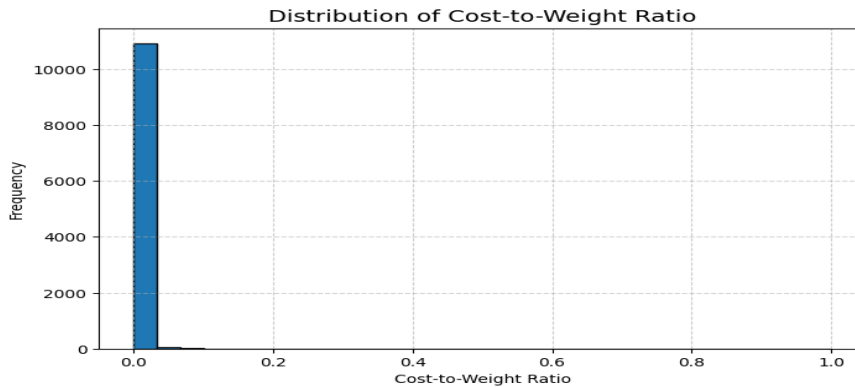
Additional Normalization Technique:

- Applied **StandardScaler** to numerical features separately for comparison.
 - StandardScaler standardizes data by removing the mean and scaling to unit variance.

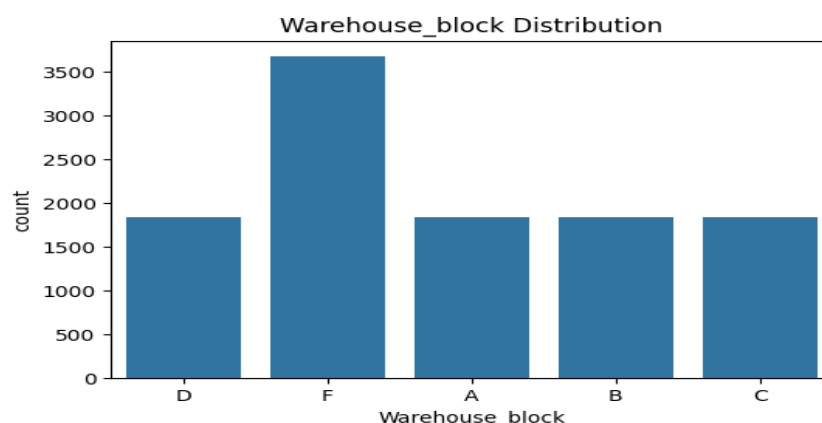
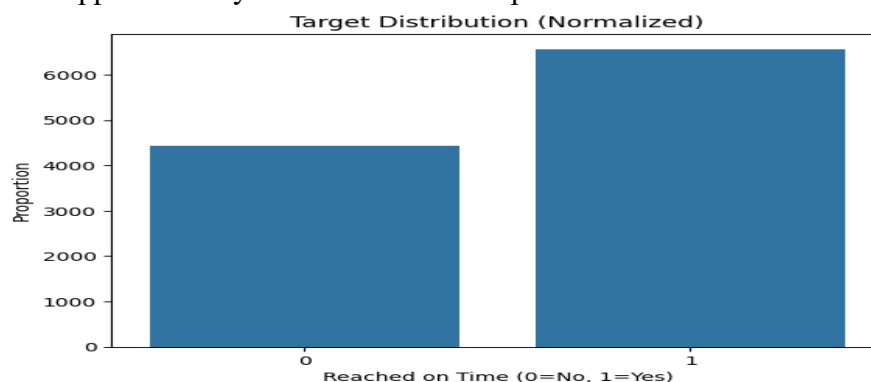
$$Z = \frac{X - \mu}{\sigma}$$

- This helps algorithms like SVM or PCA that assume normally distributed data.

Notes/Observations:



- After scaling, all numerical columns are in comparable ranges.
- No missing or infinite values remain in the dataset.
- **Value Count:**
- value_counts() counts how many samples belong to each class (0 = Late Delivery, 1 = On Time Delivery).
- The parameter **normalize=True** converts these counts into **percentages (proportions)**.
- This helps verify that the **class balance** in the training and test sets is similar to the original dataset's distribution.
- For example, if 60% of deliveries were on time in the original data, both training and test sets should have approximately 60% “on-time” samples.



11.Data Splitting

Steps Taken:

- The dataset was divided into:
 - Features (X):** All independent variables except the target column Reached.on.Time_Y.N
 - Target (y):** The dependent variable Reached.on.Time_Y.N
- Used `train_test_split()` from `sklearn.model_selection` to split data:
`X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`
 - `test_size=0.2`: 20% of data used for testing.
 - `random_state=42`: Ensures reproducibility.
 - `stratify=y`: Maintains the class distribution of the target in both train and test sets.

Results:

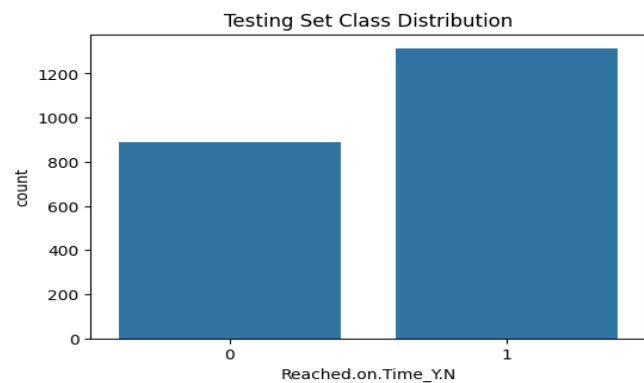
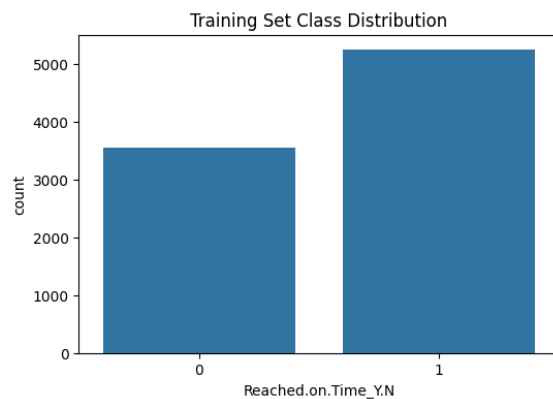
- Training Set Shape:** (8799, n_features)
- Testing Set Shape:** (2200, n_features)
- Both sets preserve the class ratio (~65% on-time deliveries).

Libraries Used:

`sklearn.model_selection (train_test_split)`

Notes/Observations:

- The dataset is now fully preprocessed, encoded, and scaled.
- Ready for **model building and evaluation**.



12. Handling Class Imbalance

Steps Taken:

- The target column Reached.on.Time_Y.N was slightly imbalanced (~65% on-time, ~35% delayed).
- To handle this, **SMOTE (Synthetic Minority Oversampling Technique)** from `imblearn.over_sampling` was used.
- SMOTE balances the dataset by generating synthetic samples of the minority class based on feature-space similarities.



Notes/Observations:

- The dataset became perfectly balanced after SMOTE.
- This prevents bias toward the majority class and improves model generalization.

13. Model Building and Evaluation

Models Used:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Gaussian Naïve Bayes
- XGBoost Classifier
- LightGBM Classifier
- CatBoost Classifier

Steps Taken:

- Created a dictionary of models with optimized hyperparameters.
- Used **Stratified K-Fold Cross Validation (10 folds)** to evaluate each model's accuracy.
- Computed mean cross-validation accuracy and standard deviation to check stability.

Cross-validation results (Overfitting check):

MODEL	CV ACCURACY	STD DEV
RANDOM FOREST	0.5944	0.0173
DECISION TREE	0.5866	0.0179
SVM	0.5863	0.0168
CATBOOST	0.5764	0.0175
LOGISTIC REGRESSION	0.5757	0.0198
LIGHTGBM	0.5734	0.0177
NAÏVE BAYES	0.5676	0.0170
XGBOOST	0.5634	0.0260
KNN	0.5460	0.0161

Notes/Observations:

- Ensemble model (Random Forest) performed best with high consistency.

14. Regularization and Overfitting Prevention

- **Regularization:** It is used to prevent overfitting, which occurs when a model learns noise and specific patterns from the training data instead of general trends—resulting in poor performance on unseen data.
- Regularization adds a penalty to large weights or overly complex trees, forcing the model to stay simpler and more general.

How to Correct Overfitting:

- Use regularization techniques (L1, L2, or tree-based regularization).
- Reduce model complexity (limit tree depth, features, or layers).
- Apply cross-validation.
- Use more data or techniques like dropout and early stopping.

Steps Taken:

- Regularization parameters were added to reduce model complexity:
 - Logistic Regression → $C=0.5$ (L2 penalty)
 - Decision Tree → limited `max_depth`, `min_samples_split`
 - Random Forest → smaller `max_features`, higher `min_samples_leaf`
 - XGBoost, LightGBM, CatBoost → smaller learning rates, subsampling, and `reg_lambda` regularization.

Result:

- The standard deviation of validation scores decreased after regularization.
- The models generalized better to unseen data with reduced variance.

Hyperparameter Tuning: It involves optimizing model parameters (like learning rate, `max_depth`, `n_estimators`) to improve accuracy and prevent overfitting.

The model's performance after hyperparameter tuning is:

Final Model Comparison Table:

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
XGBoost	0.9428	0.9713	0.94114	0.9404	0.9907
LightGBM	0.8378	0.9242	0.7326	0.8173	0.9420
CatBoost	0.8111	0.9151	0.6820	0.7815	0.9289
Random Forest	0.7367	0.9458	0.4968	0.6515	0.9114
KNN	0.6693	0.6733	0.6456	0.6591	0.7292
Decision Tree	0.6528	0.7981	0.4003	0.5332	0.7137
Logistic Regression	0.5690	0.5697	0.5301	0.5492	0.6137
Naïve Bayes	0.5768	0.6045	0.4209	0.4963	0.5761
SVM	0.5517	0.5673	0.4003	0.4694	0.5647

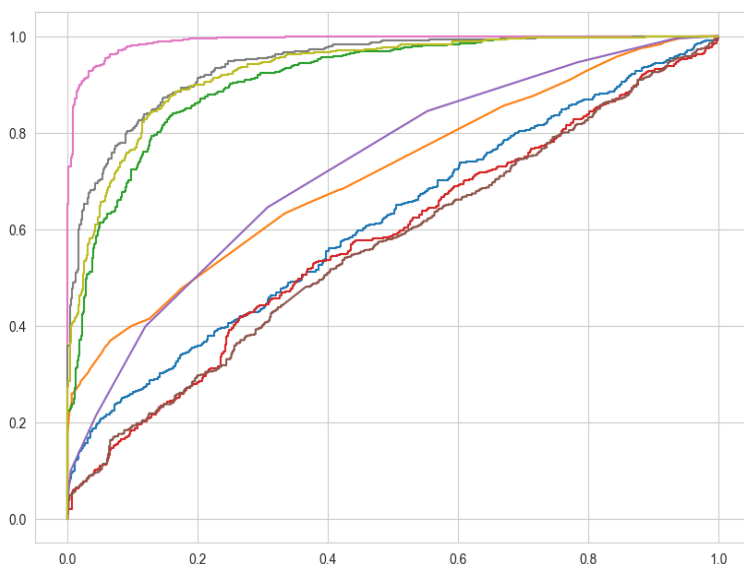


Fig: ROC_AUC Curve

Metrics Evaluated:

- Accuracy: $(TP + TN) / (TP + TN + FP + FN)$ — overall correctness of predictions.
- Precision: $TP / (TP + FP)$ — proportion of correctly predicted positives.

- Recall: $TP / (TP + FN)$ — ability to detect actual positives.
- F1 Score: $2 \times (Precision \times Recall) / (Precision + Recall)$ — harmonic mean of precision and recall.
- ROC-AUC: Area under the ROC curve — measures model's ability to distinguish between classes.

Steps Taken:

- Each model was trained on X_{train} and validated using X_{val} .
- Predictions (y_{val_pred}) were compared against actual labels (y_{val}).
- ROC curves were plotted to visualize model discrimination ability.

Notes/Observations:

Higher values of these metrics indicate better model performance, with **F1 Score** balancing precision and recall, and **ROC-AUC** showing overall classification strength across thresholds. So,

- **XGBoost Model** achieved the highest performance with

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
XGBoost	0.9428	0.9713	0.94114	0.9404	0.9907

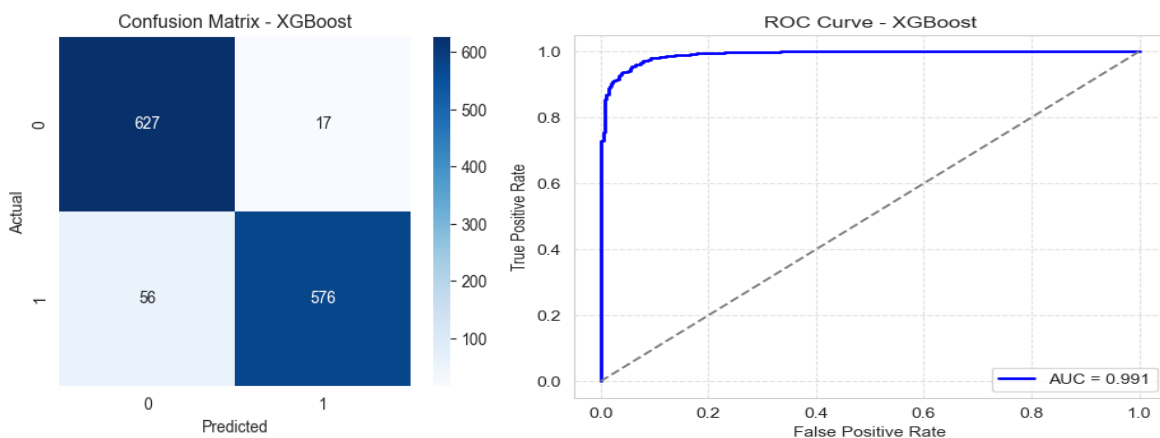


Fig: Confusion matrix & ROC-AUC curve of XGBoost

- Saved the best model XGBoost using the joblib/pickle

15. Feature Importance Analysis

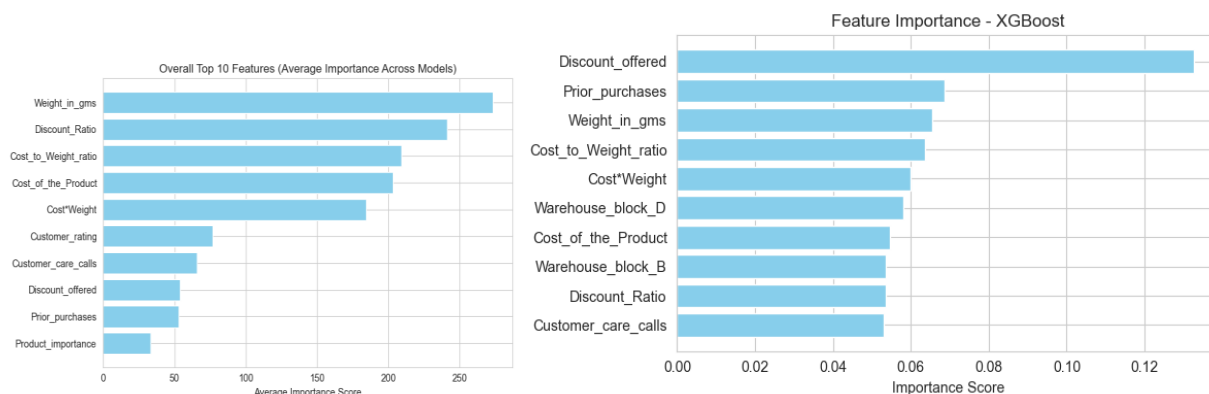
Steps Taken:

- Extracted feature importance using `feature_importances_` for tree-based models and `.coef_` for linear models.
- Computed average importance across all supported models for stability.

Top 10 Important Features:

1. Discount_offered
2. Weight_in_gms
3. Cost_of_the_Product
4. Customer_care_calls
5. Prior_purchases
6. Product_importance

7. Cost_to_Weight_ratio
8. Mode_of_Shipment_Flight
9. Warehouse_block_D
10. Gender



16. Final Model Selection and Saving

Chosen Model: **XGBoost** (Balanced Training with SMOTE)

Reasons for Selection:

- Best trade-off between accuracy and recall.
- Handles imbalanced data and nonlinear relationships effectively.
- Lower overfitting tendency due to internal regularization.

Files Saved:

- XGBoost_best_model.pkl → trained model file.
- encoder.pkl → encoders for categorical features.
- scaler.pkl → StandardScaler for numerical normalization.

Note: **Pickle** is a Python library used for **serializing and deserializing** Python objects — meaning it converts trained models, scalers, or encoders into a byte stream so they can be **saved to a file** and later **loaded back** without retraining.

17. Model Deployment Using Streamlit:

Framework Used:

Streamlit

Application Name:

ShipmentSure – On-Time Delivery Prediction

- **Streamlit** is an open-source Python framework used to **build interactive web applications** for machine learning and data science projects. It is used because it allows developers to **quickly deploy models with a simple, user-friendly interface** — without needing web development skills.

Steps Taken:

Model Loading:

- Imported the trained model and preprocessor files (**XGBoost_best_model.pkl**, **encoder.pkl**, and **scaler.pkl**) using **pickle.load()**.

User Interface:

- Created a clean and interactive form layout using **st.columns()** for better organization.
- Used input widgets such as **selectbox**, **number_input**, and slider for smooth user data entry.
- Set default input ranges and values for better user guidance (e.g., cost, rating, weight).

Feature Processing:

- Encoded categorical features (**Product_importance**, **Gender**) using saved encoders.
- Applied one-hot encoding for nominal features (**Warehouse_block**, **Mode_of_Shipment**).
- Scaled numeric features using the saved **StandardScaler** object.
- Generated new derived features (**Cost_to_Weight_ratio**, **Cost*Weight**) dynamically during prediction.
- Handled missing and infinite values by replacing them with zero to ensure smooth processing.

Prediction and Threshold:

- The model predicts the probability of on-time delivery using **model.predict_proba()**.
- A **fixed decision threshold of 0.45** was used for classification.
- If the probability $\geq 0.45 \rightarrow$ the shipment is predicted as **“ON TIME”**, else **“DELAYED”**.

Output Display:

- If prediction $\geq$ threshold $\rightarrow$ **“Shipment ON TIME”** (Green Success Message)
- Else $\rightarrow$ **“Shipment DELAYED”** (Yellow Warning Message)
- Displays confidence score (probability) for transparency.

Code Snippet:

```
if prediction == 1:
    st.success(f"**Prediction: SHIPMENT WILL REACH ON TIME!**")
    result_text = "The model predicts the shipment will be delivered successfully on time. This is a low-risk shipment."
else:
    st.warning(f"**Prediction: SHIPMENT WILL BE DELAYED.**")
    result_text = "The model suggests this shipment is likely to be delayed. **Immediate action is recommended.**"
st.info(f"Confidence Score (Probability of On-Time): **{prob*100:.3f}%**")
st.write(result_text)
```

streamlit UI interface and Output:

 **ShipmentSure: On-Time Delivery Prediction**

Enter shipment details below to predict whether the delivery will be on time or delayed.

Warehouse Block C	Prior Purchases 9
Mode of Shipment Road	Product Importance high
Customer Care Calls 3	Customer Gender F
Customer Rating 5	Discount Offered (%) 10
Cost of Product (\$) 250	Weight of Product (grams) 1500

Predict Delivery Status

Prediction: SHIPMENT WILL REACH ON TIME!

Confidence Score (Probability of On-Time): 69.584%

The model predicts the shipment will be delivered successfully on time. This is a low-risk shipment.

 **ShipmentSure: On-Time Delivery Prediction**

Enter shipment details below to predict whether the delivery will be on time or delayed.

Warehouse Block B	Prior Purchases 2
Mode of Shipment Flight	Product Importance high
Customer Care Calls 2	Customer Gender M
Customer Rating 5	Discount Offered (%) 8
Cost of Product (₹) 600	Weight of Product (grams) 1101

Predict Delivery Status

Prediction: SHIPMENT WILL BE DELAYED.

Confidence Score (Probability of On-Time): 50.083%

The model suggests this shipment is likely to be delayed. Immediate action is recommended.